Reversium by 3Sci Labs

Start here

What Reversium helps you do

Reversium reads a software package and organizes supported source and binary clues into findings, diagrams, evidence and explicit gaps. Use it when receiving a handover, inheriting unfamiliar code, investigating a feature or preparing maintenance. Ordinary browser analysis does not install, run or certify software; the separate experimental local sandbox tests one reviewed build/start profile.

Choose one useful outcome

For an owner: identify what was delivered and agree on one feature to check. For a developer: find likely entry points, dependencies and source locations. For a maintainer: map possible affected components before changing them. A useful outcome is a short, evidence-backed checklist rather than a claim that the entire platform is understood.

A first session in five steps

1. Open the browser version and choose Start exploring. 2. Choose Understand the software, Prepare a developer handover, or Plan maintenance or modernization. 3. Continue to ZIP, select your source-code package and choose Analyze ZIP locally. 4. Read Review findings, then inspect one Architecture workflow and its Evidence. 5. Read Coverage and Save & next steps; save a report and assign the remaining checks.

Finding help

Help is available on Start and in the report tabs. Search by task or keyword, such as handover, evidence or sandbox. Matching chapters open automatically; clear the search to show every chapter again. Download user guide (PDF) keeps an offline copy. The guide shows its build number and is rebuilt with each release.

Example questions

Where does the save action lead? Which services are referenced? Which files should a new developer read first? What configuration, permissions and sample data still need confirmation? Reversium provides investigation leads; isolated installation and runtime tests answer whether those leads work.

Online and local editions

Online browser edition

Open the browser edition from the Reversium product page. No installation, account or payment is needed for analysis. The app downloads its self-hosted Python runtime, then reads the selected ZIP in a browser worker on your device. Use a current desktop browser. The first visit downloads about 14 MB of app/runtime assets and can take longer.

Local application

Use the local application package supplied by 3Sci Labs. Install Python 3.10 or newer, open a terminal in the project and run python -m reversium. Open the local address printed in the terminal; it can use a free port. On Windows, Start-Reversium.ps1 is an alternative launcher. Stop the server with Ctrl+C. The base analyzer has no third-party Python dependency; optional AI has separate requirements.

Main differences

Both editions use the same analysis engine, findings and core diagrams. Online: 32 MiB compressed ZIP limit, five-minute worker stop, browser-memory constraints, no embedded cloud account connection. Local web application: 64 MiB upload limit, local Python process and existing optional ChatGPT account connection. Local command line: report exports and baseline comparison without a web page; archive safety limits still apply.

AI differences

Online AI is a manual handoff: select a provider and evidence, review/edit the exact request, download it, then decide whether to paste and submit it on that provider website. It is not an account connection or embedded chat. Local AI offers an account connection inside Reversium; install requirements-ai.txt if needed, follow its connection wizard and review sharing before each request. Eligibility, network access and provider limits apply. A successful sign-in does not guarantee a successful model response.

Which should I use?

Start online for a quick inspection without installing anything. Use local for larger supported packages, repeatable command-line work or the existing optional account integration. Use neither as proof that a platform is production-ready. Online remains free to analyze; optional AI services follow the terms and limits of the account you choose.

Selecting a package and handling files

Choose the best input

Prefer the project source-code package with manifests and migrations. An installer or binary-only archive provides fewer clues. Do not include unrelated backups, generated dependencies or large media if they are unnecessary. Keep the original delivery intact and analyze a separate copy when preparing a smaller archive.

What stays on the device

The online app does not upload the ZIP. It holds the file and report in page/worker memory, not app-managed localStorage or IndexedDB. Static web hosting receives normal app and asset requests, including connection information such as IP address; those requests do not include ZIP contents. No input source is executed. No AI evidence is submitted automatically.

Clearing and saving

Cancel and clear stops the worker and returns to Start. Clear and start again removes the report and selected-file references. Closing the page also releases app references; this is not a secure-memory-erasure guarantee. Browser caches may retain application assets. Downloaded reports and requests remain wherever you save them and can contain private filenames or code-derived details. Share them deliberately.

Limits and rejected archives

Online input is limited to a non-empty ZIP no larger than 32 MiB; the local web application accepts up to 64 MiB. Analyzer bounds include 10,000 files, 256 MiB expanded total and 4 MiB content inspection per file, plus compression, path and graph bounds. Encrypted, damaged, unsafe or oversized archives may be rejected or only partly inspected. Nested containers receive bounded directory inspection, not recursive analysis. Read the actual Coverage output.

Reading the main report

Header and inventory

The logo and build number identify the report generator. Main tabs stay near the top. Technical inventory numbers remain expanded: files, bytes, binary candidates and findings. These are counts of inspected material, not a completeness, quality or security score.

About and Review findings

About explains Reversium itself. Review findings describes supported clues about the uploaded software in plain language. Open Why we say this to inspect the evidence behind a statement. Detailed findings for a developer expands more technical explanations. Different starting goals alter guidance, not the underlying source evidence.

Evidence

Use the search box to filter findings by file, symbol, value, kind or resolution. A source link from a feature opens Evidence and reveals its row. Locations are file and line references, or byte offsets for binary clues; they are not an in-browser source editor. Traced means a supported static reference was found. Inferred means a candidate relationship needs confirmation. Neither means the program ran successfully.

What to do with unknowns

Record the question, the relevant finding, the person who can resolve it, and a concrete verification step. For example: confirm the API route and database table by running one isolated user journey with test data and appropriate permissions. Missing references can reflect unsupported syntax or missing files, not missing functionality.

Architecture and relationship diagrams

Subsection buttons

Open Architecture and choose Workflow, Graphical feature map, Feature catalogue, Platform profile, Supporting services, or Imports & dependencies. The selected subsection replaces the other views, so you do not need to scroll through the whole architecture report. Available sections depend on report content.

Workflow

Choose a screen and data branch. Read the ordered path from screen through callbacks and an API to a database operation. Each step links to its evidence. The view follows a bounded candidate path; it can show a logging side effect rather than the main purpose of a request. It does not establish runtime conditions, arguments, permissions or successful responses. Read What remains unknown.

Graphical feature map

Select a feature to focus the map. Click a node or connection to inspect participants, evidence and unknowns. Solid lines represent traced source references; dashed lines represent inferred connections. Shared names do not prove a shared database. The display is capped at 120 nodes and 240 edges per view, with omissions disclosed. Download map SVG saves the drawn map; JSON retains the full generated graph, which is itself bounded.

Feature catalogue and file relationships

The catalogue lists supported feature candidates, operations and evidence. Imports & dependencies draws file-to-local-module candidates or referenced packages; it shows the first 100 unique edges and is not a complete runtime call graph. Full available findings remain in Evidence. Platform profile lists candidate technologies and runtimes. Supporting services lists references that may be optional, development-only or environment-dependent.

When a map looks incomplete

Filter to one feature and inspect its evidence. Check Coverage for unresolved targets and unsupported syntax. Dynamic code, aliases, middleware, component instances, conditions and runtime configuration can break a static path. Do not infer that an absent edge proves there is no relationship. Large diagrams are an investigation aid, not a complete specification.

Coverage, binary clues and readiness

Coverage subsection buttons

Overview describes what inspection can establish. Binary inspection shows supported metadata readers and target/runtime clues. File inventory lists path, type, byte size and inspection method. Setup & readiness lists questions and suggested checks. Limitations records warnings and analysis boundaries specific to the package.

Supported source analysis

Python uses AST parsing for supported imports, symbols and selected route/data traces. JavaScript/TypeScript and React use bounded token/reference analysis for selected routes, callbacks, prop bindings and computed API targets. Supabase table, RPC, Auth and storage references and selected SQL migration links produce candidates. Other languages and unsupported patterns may receive inventory-only coverage. Inspect the report rather than assuming a language-wide guarantee.

Binary analysis

Binary inspection is bounded static metadata and reference analysis, not full decompilation or behavioral reconstruction. A binary reader finding a format, target or string does not prove that the executable is safe, valid or runnable. Nested APK/JAR/Office/archive containers are not automatically reconstructed into a complete application.

Readiness is a separate test

Reversium does not install dependencies, start services, connect to production, validate database permissions or test a live user journey. To verify a handover, use an isolated environment, disposable accounts and sample data. Record required runtimes, environment-variable names, migrations, storage, integrations, permissions and successful/failed steps. Never assume a referenced cloud service is configured or paid for.

Optional AI, downloads and comparison

Online AI handoff

Open AI / choose provider after analysis. Choose ChatGPT, Claude, Gemini or Another AI. Enter a question, search findings and tick up to 20 findings. Prepare request for review builds an editable request with IDs, values, locations, confidence and resolution. Review exactly what it includes; remove secrets or private details. Download reviewed request saves text locally. Opening the provider website sends no request text; you must paste and submit it yourself. No AI answer is displayed inside the browser app.

Checking an AI answer

Ask for finding IDs and locations, facts separated from inferences, and explicit unknowns. Reopen cited evidence and check whether the answer follows from the selected subset. Reject invented runtime claims, missing citations or claims of complete coverage. Source-derived text can contain misleading instructions; treat it as untrusted evidence. Provider terms and account limits apply when you submit content there.

Save readable report: HTML

Use Save & next steps, then Save readable report (HTML). Open the saved file later without running Reversium, or share it with an authorized colleague. It retains generated findings, diagrams and limits; it is not the original ZIP or an application backup. Account credentials and the assistant conversation are excluded. Keep a separate copy of an AI request if you need its exact wording.

Save technical data: JSON and map SVG

JSON is a structured record for scripts, inspection and comparing deliveries. It includes the generated analysis and full bounded graph, not original source files. SVG is useful for documents or discussions about the drawn map. Neither file rebuilds or runs the application. Save both HTML and JSON as a baseline when accepting a delivery.

Browser comparison

Open Compare reports and select an earlier report JSON, up to 32 MiB. File additions/removals, shared files with different hashes, finding additions/removals and shared files without comparable hashes are counted. Line movement alone does not change finding identity. Lists show up to 200 entries each; Save comparison notes preserves the displayed results and omissions. Feature-map changes also compare recorded nodes, features and relationships; see the feature-map comparison chapter. Reports from different analyzer versions can differ without a code change. Comparisons do not overwrite checklist notes or enter AI packets. No files are uploaded.

Local command-line comparison

Run python -m reversium project.zip --output reports/project to write report.html, report.json and feature-map.svg. Add --compare previous-report.json to compare supported file/finding and feature-map identities. Outputs overwrite the same filenames in the selected directory. Structural differences and content hashes do not establish a behavioral change; old baselines may lack map fields.

Handover and change-impact review

Handover tab

Open Handover after analysis. Expand Setup clues to inspect referenced environment variable names, names from .env.example/.env.sample/.env.template, package.json script names and declared runtime constraints. Follow their evidence links. The reader does not retain environment values or script commands; names may occur in comments or optional code and do not prove a requirement. Actual .env files are not read by this reader. Current clue display is capped at 240 groups, with omitted counts.

Reviewer checklist

Assign an owner and status to each action and record a verification reference in Notes. Verified by reviewer is your statement, not automated proof. Do not enter credentials or sensitive values. Save the handover checklist JSON for structured records, or save readable handover notes for a colleague. These files can include private review notes. They are not uploaded, included in AI evidence requests or preserved by ordinary report exports. Clearing/closing loses unsaved annotations. Restore saved checklist accepts JSON up to 1 MiB. New checklists use an exact ZIP SHA-256 fingerprint (available for archives up to 64 MiB); repacking changes it. Review the preview and explicitly replace current notes. Legacy checklists only match package names and require independent confirmation; no exact identity is claimed. Invalid or mismatched inputs leave notes untouched.

Change impact tab

Choose a file to find code nodes assigned to that source file by the supported reader. The view explores upstream and downstream graph directions up to three hops, 120 candidate nodes per direction and 60,000 edge checks per direction. These are investigation candidates, not proven affected components. Starting nodes use explicit source-file metadata rather than cross-file evidence locations. File association does not prove runtime execution. Dynamic and omitted references can hide relationships; inferred edges can overstate them. Use source links and isolated tests before estimating or changing software.

Choosing the right tool

Reversium is a private, model-free first step for ZIP-based software handover. Specialist tools offer deeper code quality rules, security queries or architecture workflows. The public market comparison explains the documented focus of CodeBoarding, SonarQube, CodeQL and Joern, along with Reversium limitations and planned work. It is not an exhaustive market survey or product benchmark. No model or paid service is needed for the new checklist and graph views.

Compare feature maps between deliveries

Start with two reports

Analyze the current ZIP, open Compare reports and select an earlier report JSON. Both reports stay on your device. Feature-map changes appear before the file comparison. Reports without a map show comparison unavailable; missing coverage is not interpreted as removals.

Read the changes

Counts show added, removed and changed features, nodes and relationships. Stable IDs are compared. Relationship changes include endpoints, label, confidence and recorded feature membership; evidence line movement alone is ignored. A renamed identity may appear as a removal plus an addition. Analyzer improvements and differences in coverage can change maps even when software is unchanged.

Follow evidence

Each relationship card shows earlier and current source/target names, connection labels and confidence, plus a small diagram. Current evidence links open current findings. Earlier source locations are labelled and must be checked in the saved earlier report: their finding numbers must not be interpreted against the current report.

Decide what to test

Directly recorded feature membership is shown separately from a broader neighborhood, which may contain unrelated features through shared nodes. Explore these as candidates, not proven effects or failures. Inspect the source and run isolated workflow tests before changing or estimating software. No candidates found does not prove no effect.

Bounds and saving

Imports retain the 32 MiB report limit. Graphs allow up to 20,000 nodes, 40,000 edges and 1,000 features. Invalid identities, endpoints and evidence references are rejected. The view shows up to 100 changes per node/feature list and 100 relationship cards, with omitted counts. Each relationship side explores at most two hops, 120 nodes and 200 edge visits. Feature names show up to 20 per list and evidence up to four references. Save comparison notes preserves displayed results and omissions. Checklist notes are preserved, and no comparison is sent to AI.

Troubleshooting and a practical handover

Runtime cannot start or analysis is slow

Allow the initial app/runtime download to finish and use a current desktop browser. A network interruption, unsupported browser or device memory limit can prevent analysis. Cancel and retry a smaller source-only package, or use the local application. Online analysis stops after five minutes. Do not repeatedly reload a report you have not saved.

Few findings or an empty workflow

Check that the package includes source files and manifests. Read File inventory to see which files were content-inspected versus inventoried only. Read Limitations for unsupported syntax and unresolved links. Binary-only or incomplete source deliveries cannot establish a full platform map.

An evidence link or download is unclear

A source link selects an Evidence row; it does not fetch a repository file. Downloads normally go to your browser download folder or trigger a save dialog. Check browser download controls if a file is missing. HTML is for reading; JSON is for tools; SVG is the drawn diagram; the AI request is only selected text you reviewed.

AI connection or response fails locally

Check the local assistant wizard, optional dependencies, network connection, account eligibility and provider usage limits. Review the error without posting credentials. Signing in and checking connection do not themselves share the ZIP. Do not assume a successful connection proves inference availability; there is no automatic paid API fallback. Online has no live account connection to repair.

A handover you can act on

Choose one important feature and its expected result. Save the HTML and JSON. Give the developer its workflow/evidence and a list of missing services, configuration and data. Agree on an isolated setup test and record the outcome, date and owner for every gap. Decide what is ready, what is blocked and what must be tested next. Keep the report with the exact delivery it describes.

Glossary

API: a request interface between software components. Callback: a function passed to run when an event occurs. Dependency: another package referenced by the code. Manifest: a file listing project settings or packages. Migration: a script describing a database change. Static analysis: reading files without executing the uploaded program. Evidence: the recorded source location or metadata that supports a finding. Unknown: a claim the available inspection has not established.

File alerts: what the colors mean

Read the signals

Green means no signals from the supported checks, not guaranteed safe. Yellow means possible damage, active content or incomplete inspection. Red means a potentially dangerous pattern or filename disguise requiring review, not confirmed malware. Follow each evidence link and read coverage. A ZIP integrity check is not a malware scan.

Act on the result

Ask for a replacement for damaged files and ask a developer to explain suspicious patterns. Reversium does not automatically execute, delete or quarantine your files. It is not antivirus or a complete vulnerability audit. Do not use a green light as permission to run unknown code.

Sandbox: real execution, limited support

Its purpose

A sandbox is a disposable place to test whether a reviewed code package actually builds, starts and completes a chosen task, without using live accounts or production data. Static analysis reads code. The experimental local sandbox executes real code and records observations; it does not simulate the application or invent successful behavior.

What is available now

The online browser edition never runs your code package. The local Windows QEMU pilot supports one fixed, developer-reviewed example application profile. It is not a general Run ZIP button. A developer must review and adapt the preparation, dependencies, commands and acceptance checks for another project before claiming support. A matching language or framework does not make your ZIP compatible.

Isolation and test data

A fresh trusted Linux guest is checked before application input. No guest network adapter, host shares, clipboard integration or forwarded network ports are enabled. The fixed service profile runs real disposable database, login and storage services with synthetic accounts. A guarded local browser preview relays bounded requests over serial. It can be slow; external services are intentionally unavailable. No VM guarantees zero risk.

What a result establishes

Record the exact ZIP fingerprint, tested filtered-copy fingerprint, profile, date, pass/fail and missing checks. Separate setup success, API tests and actual browser clicks. These are unsigned editable reviewer records, not certification. A successful build is not a working platform, and a successful task is not production readiness.

Stop and retained files

Use the protected local Stop control or the supervised deadline. Guest database and files are temporary. Private input images, package caches, logs, screenshots and browser sessions can remain on the host. Keep them private and retain only what you need; stopping is not a secure-erasure guarantee.

Working alongside your development tools

Visual Studio, VS Code and other editors

Use Reversium before or alongside your editor: inspect the package, follow evidence to file/line locations, then open the original source in your normal editor. Reversium does not edit or synchronize source, install editor extensions or provide native debugging integration.

Version control and repository hosting

Export a source-code package from your repository service, including GitHub, and analyze it locally. Save report JSON as a baseline; compare a later delivery. Reversium does not connect repository accounts, watch branches, create pull requests or push your uploaded project.

Supabase and other backends

Selected Supabase table, RPC, Auth, storage and SQL references can become static clues. A developer still checks project settings, permissions, migrations, environment values and data. Analysis does not log into or validate your live backend.

Docker and Railway

Supporting container and deployment files can contribute readiness clues. A developer uses Docker or Railway separately to build, test or deploy where appropriate. Reversium does not launch Docker containers or deploy to Railway. Their accounts, hardware, terms and costs are separate. A container is not the experimental sandbox boundary.

AI coding tool handovers

For example, export your authorized code from a project developed with Claude, analyze it, and give a developer using Codex the code package, evidence-backed report, saved checklist and selected context. This is a manual handover, not automatic tool conversion. Conversations, hidden instructions, agent memory, credentials and account sessions do not transfer.

Authorized code understanding and reverse engineering

How to describe the product

Reversium supports authorized code understanding and reverse-engineering analysis of code structures and relationships. It reads supported source and binary clues; it does not fully decompile or recreate a platform. Use packages you own or are authorized to inspect.

Rights and limits

Reverse engineering is not universally permitted in every situation. Copyright, licences, confidentiality, privacy and access-control laws depend on jurisdiction and purpose. Permission to possess a ZIP is not necessarily permission to publish its contents. Reversium does not grant rights or bypass encryption or access controls. Obtain jurisdiction-specific advice for restricted third-party work.

Examples of legal distinctions

US copyright law has a limited interoperability exception under 17 USC 1201(f), alongside restrictions on circumvention. UK law distinguishes observing/studying/testing by a lawful user under section 50BA from conditional decompilation under section 50B. Those provisions are not blanket permission. The public Product guidance document links the sources and explains the positioning.

Feature list and seven visual journeys

Always current with the release

Download the feature list for an edition-by-edition explanation of current functions and limits. The release build generates it from the reviewed product catalog and the release gate checks its version, source fingerprint and artifact hashes. When functionality changes, update the catalog before building Help; the check prevents stale generated assets, not inaccurate editorial claims.

Choose a practical route

The seven diagrams cover understanding a delivery, handing over between AI coding tools, taking over maintenance, planning a change, comparing deliveries, asking AI with selected evidence and verifying behavior in a reviewed local sandbox. Each shows steps, a useful output and a boundary. The sandbox diagram is conditional, not a promise to execute your ZIP.

Where to find them

The graphical journeys appear below the chapters in Help, including on Start before analysis. Download all seven as one PDF. The feature list and journeys contain public documentation only; no code package, report, account or private findings are included.

Developer appendix: experimental local preparation

Compatibility first

These commands retain a fixed reviewed example application profile. They are not a setup recipe for an arbitrary code package. Review manifests, expected directory structure, build commands, migration policies and sample-data behavior before use. Unsupported projects need a separate reviewed implementation.

Prepare trusted assets

With existing compatible Windows hardware and 7-Zip, run python scripts/bootstrap_qemu_lab.py. Then use python scripts/prepare_project_offline.py "path/to/your-package.zip" for the compatible reviewed profile. Public dependency identifiers are downloaded, not source contents. Packages are not installed or project scripts executed on the host. Signed Alpine package trust and locked dependency integrity remain enforced.

Test before application input

Use python -m reversium --sandbox-self-test, then the explicit python -m reversium --sandbox-project "path/to/your-package.zip" action. Software emulation is the default; --sandbox-accelerator whpx is optional only where already enabled. The build/start profile repeats the blank-guest isolation gate. No automatic network fallback is enabled.

Disposable services and browser preview

For the supported profile, cache public service assets with python scripts/prepare_project_services.py. The --sandbox-project-services "path/to/your-package.zip" action performs the reviewed API test. The separate python scripts/test_project_cms.py "path/to/your-package.zip" command exposes a protected loopback preview. Account and entry files stay private. Open /__reversium on that preview origin to stop it.

Bounds and record linking

The base profile uses 1 GiB guest RAM; service and CMS profiles use 3 GiB and two virtual CPUs with a bounded 2 GiB workspace. Automated application tests allow ten minutes; browser previews allow twenty minutes within thirty minutes total. The serial relay limits HTTP bodies/responses to 2 MiB. In Setup & readiness preview the portable run-evidence.json for the exact original ZIP fingerprint, then explicitly attach it. A matching record does not authenticate its producer.

Reading a pilot result without overclaiming

Observed behavior belongs to one delivery

A reviewed example package built and completed selected API and text-content browser checks in the isolated guest. These observations are historical tests of that package, not capabilities guaranteed for your ZIP. Original fingerprints, dates, outcomes and blockers remain in the separate anonymized test reports.

Image test example

A small synthetic PNG passed missing-rights and missing-credit validation, uploaded successfully and was attached to a submitted content package. A separate reviewer saw an empty queue in the same organization. Image approval, publication and anonymous display therefore remain unverified. Do not replace a blocked screen with a database edit and call it a browser success.

Repeatable test approach

Use synthetic data and new disposable accounts. Record each screen, action and visible outcome. Review the whole image and its real rights/credit before approval; after publication check the actual visitor image and attribution. Large images, formats, metadata, security and production durability need separate tests. Slow file-picker automation and serial transport are known practical blockers.